

Rate Limiter Engine

A C++ implementation and benchmark comparison of three production rate-limiting algorithms, unified behind a common Strategy-pattern interface.

Motivation

Rate limiting is a core building block of real backend systems — APIs, login endpoints, and payment gateways all need to cap how many requests a client can make in a given time window, both to protect infrastructure and to prevent abuse. This project implements three widely-used algorithms from scratch, compares their trade-offs, and demonstrates a well-known flaw in the simplest one experimentally.

Design: Strategy Pattern

All three algorithms implement a common `RateLimiter` interface with a single method, `allowRequest(clientId, timestamp)`. This means any algorithm can be swapped in or out of a calling system without changing any other code — a direct application of the Open/Closed Principle and the Strategy design pattern.

```
class RateLimiter {
public:
    virtual bool allowRequest(const std::string &clientId, long long nowMillis) = 0;
    virtual std::string name() const = 0;
};
// TokenBucketLimiter, SlidingWindowLogLimiter, FixedWindowCounterLimiter
// each implement this interface independently.
```

Algorithm 1: Token Bucket

Each client owns a bucket holding up to *capacity* tokens, refilled continuously at a fixed rate. Each request consumes one token; if the bucket is empty, the request is denied. This allows short bursts (up to the bucket size) while enforcing a long-run average rate — the algorithm actually used by AWS API Gateway and Stripe's API.

Algorithm 2: Sliding Window Log

Maintains an exact timestamp log per client. On each request, timestamps older than the window are evicted, then the request is allowed only if the remaining log is under the limit. This is exact (no boundary artifacts) but costs $O(\text{window size})$ memory per client.

Algorithm 3: Fixed Window Counter

Divides time into fixed windows (e.g. every 1000ms) and keeps a single counter per client that resets each window. Extremely cheap ($O(1)$ memory, $O(1)$ time) but has a well-known flaw: a client can send the full quota right at the end of one window and the full quota again right at the start of the next, achieving up to 2x the intended rate in a short burst around the window edge.

Experiment: Reproducing the Boundary-Burst Flaw

To verify this isn't just theoretical, the engine sends 10 requests at $t=990-999\text{ms}$ (end of window 1) and 10 more at $t=1000-1009\text{ms}$ (start of window 2), against a limit of 10 requests/window, for both Fixed Window

Counter and Sliding Window Log:

Algorithm	Allowed out of 20	Result
Fixed Window Counter	20 / 20	Exceeds intended limit (2x)
Sliding Window Log	10 / 20	Correctly enforces the limit

This confirms the textbook trade-off experimentally: Fixed Window Counter is cheap but exploitable at window boundaries; Sliding Window Log is exact but costs more memory per client.

Throughput Benchmark (measured on this machine)

20,000 simulated requests across 200 clients were run through each algorithm to measure raw processing speed of the rate-limiting logic itself (no network I/O):

Algorithm	Allowed	Denied	Time (ms)	Requests/sec
Token Bucket	7,796	12,204	1.64	~12.2 million
Sliding Window Log	7,980	12,020	1.78	~11.2 million
Fixed Window Counter	8,000	12,000	1.68	~11.9 million

All three algorithms process well over 10 million simulated decisions/sec purely as in-memory logic, confirming $O(1)$ -per-request behavior (the small differences reflect hashmap and deque overhead, not algorithmic complexity differences).

Tech Stack

C++17, STL containers (`unordered_map`, `deque`), compiled with `g++ -O2`, `std::chrono` for benchmarking, abstract base class for the Strategy pattern.

Possible Extensions

Distributed rate limiting using Redis (shared state across multiple server instances), a Sliding Window Counter hybrid (approximate but $O(1)$ memory, used by Cloudflare), and wrapping the engine behind an actual HTTP middleware layer.